MCP Bridge

Connector Submission Guide

For service owners who want their API available on Alexa+ · Manifest version 1.0

Prototype built for the Amazon Developer Hackathon 2026 (Alexa+ track). All URLs, accounts and review steps are demo values.

> **In short:** describe your API in one JSON file (your OpenAPI spec plus a few voice fields), host a public copy on your domain, and upload the full file in the Owner Portal. After an Amazon reviewer approves it, any Alexa+ user can add your service from the mobile app and log in with their own account on your site. We build and host the MCP server for you.

1. What you submit

- **One JSON file** named `alexa-connector.json`. It holds your publisher details, how users log in (OAuth 2.0), a few private values for the reviewer, and your full OpenAPI spec with an `x-alexa` block on every endpoint.
- **Optional supporting PDF** with anything else the reviewer should know (screenshots, business rules). Your JSON must still be complete on its own.

2. Before you start

- **An OpenAPI 3.1 spec** for the endpoints you want on Alexa (3.0.x is also accepted). Expose 20 endpoints or fewer and choose the ones people would actually ask for by voice.
- **An OAuth 2.0 client** on your auth server for MCP Bridge, using the authorization code grant. Register this redirect URI: `https://bridge.example.com/oauth/callback` (demo value; the Owner Portal shows the exact one). Support refresh tokens and, ideally, PKCE and token revocation.
- **A test account** on your service that the reviewer can log in with. It should have realistic data (a past order, a saved address) so every endpoint can be tried.
- **HTTPS everywhere.** Your API, auth server and policy pages must all be on your domain or its subdomains.

3. File structure

The file has five top-level fields. Everything except `private` is public.

```
{
  "manifestVersion": "1.0",
  "publisher":      { ... },   // who you are
  "accountLinking": { ... },   // how users log in to your service
  "private":        { ... },   // upload only: never put this in the hosted copy
  "openapi":        { ... }    // your OpenAPI spec, with "x-alexa" on each operation
}
```

publisher

Field	Required	What to put
`name`	Yes	The name users will say, e.g. "Demo Bistro". Keep it short and easy to pronounce.

Field	Required	What to put
`domain`	Yes	Your website's domain without https:// or www, e.g. demobistro.test. Users search for this in the app.
`description`	Yes	One or two sentences on what users can do. Shown in the app and to the reviewer.
`category`	Yes	One of: food_and_drink, shopping, travel, productivity, finance, health_and_fitness, entertainment, home_services, other.
`logoUrl`	Yes	Square PNG, at least 512×512, on your domain.
`privacyPolicyUrl`, `termsUrl`	Yes	Public pages on your domain.
`contactEmail`	Yes	Where the reviewer can reach you.

accountLinking

Each Alexa+ user logs in on **your** login page. MCP Bridge never sees or stores their password. It stores only the access token, refresh token, expiry and scopes, encrypted, and deletes them when the user unlinks.

Field	Required	What to put
`type`	Yes	Always "oauth2_authorization_code" in version 1.0.
`authorizationUrl`	Yes	Your OAuth authorize endpoint.
`tokenUrl`	Yes	Your token endpoint. It's used for both the first exchange and refreshes.
`revocationUrl`	No	Called when a user unlinks your service. Strongly recommended.
`clientId`	Yes	The client ID you created for MCP Bridge. This is not secret.
`tokenAuthMethod`	No	"client_secret_post" (default) or "client_secret_basic".
`scopes`	Yes	The smallest set of scopes your x-alexa endpoints need.
`pkce`	No	true if your auth server supports PKCE (S256). Recommended.

private (upload only)

Field	Required	What to put
`clientSecret`	Yes	The secret for the client ID above. It goes into an encrypted secrets store.
`testAccount.username`, `.password`	Yes	The reviewer's login on your service.
`reviewNotes`	No	Anything the reviewer needs, e.g. "order ord_1001 exists for status lookups".

> **Never host the private block.** The copy you publish on your website must be the same file with `private` removed. Domain verification compares the two while ignoring `private`.

4. Your OpenAPI spec and the x-alexa block

Put your full spec under `openapi`. MCP Bridge turns each operation into one tool that Alexa+ can call. The spec rules below exist so Alexa+ picks the right tool and fills in the right inputs.

- `servers[0].url` is your production API base URL, on your domain.
- Every operation has a unique `operationId` made of letters, digits, _ or - (64 characters max), e.g. `place_order`. This becomes the tool name.
- Every operation has a plain-language `description` that says what it does and when to use it. Alexa+ reads this to choose a tool, so it matters more than anything else you write.
- Every parameter and request-body property has a `description` and a precise schema. Use `enum`, `default`, `minimum`/`maximum` and `maxLength` wherever they apply.
- Describe your success response with a schema. The fields you name in `speakableResponse` must exist in it.
- Use only JSON request and response bodies. File uploads and streaming aren't supported in version 1.0.

x-alexa fields (on every operation)

Field	Required	What to put
`examplePhrases`	Yes	2 to 5 things a user would actually say, each naming your service: "order a paneer wrap from Demo Bistro".
`sideEffect`	Yes	"none" (only reads data), "write" (changes something), or "payment" (charges money).
`requiresConfirmation`	Yes	true makes Alexa ask before calling. It must be true for payment and is strongly recommended for write.
`confirmationPrompt`	If confirming	The question Alexa asks. It can use the tool's inputs: "Should I place this {fulfillment} order?"
`speakableResponse`	Yes	What Alexa says on success, filled from the response body. Keep it to one or two short sentences.
`emptyResponse`	No	What to say when a list in the response comes back empty.
`errorResponses`	No	Map of HTTP status → sentence, e.g. { "409": "Sorry, that's sold out." }. Other errors get a generic apology.

Response templates

Placeholder	Becomes	Example
`{field.path}`	The value at that path in the response (dot notation).	"ready in {etaMinutes} minutes"
`{list:path}`	Up to three values joined naturally: "A, B and C".	"including {list:items.name}"
`{count:path}`	The number of items in an array.	"{count:items} dishes"

Writing for voice

- **Say it out loud.** If a response takes more than about 8 seconds to say, shorten it. Alexa+ can always ask for more.

- **Return spoken-ready fields** where it helps, like `totalFormatted`: "$14.50" rather than 1450 cents, or `statusLabel`: "being prepared" rather than PREPARING.
- **Don't read IDs aloud.** Let users refer to "my latest order" by supporting a value like `latest`.
- **Vary your example phrases.** Mix questions and commands, and include the service name in each one.

5. Host the public copy

Publish the file with `private` removed at:

```
https://<your-domain>/.well-known/alexa-connector.json
```

- Serve it over HTTPS with `Content-Type: application/json` and no login required.
- It proves you control the domain, and it's how the Alexa app finds your service when a user searches for your website.
- Whenever you change the file, update the hosted copy and upload the new version in the Owner Portal. Changes go through review again.

6. Test it in Postman

- In Postman choose **Import**, then **Raw text**, and paste only the value of the `openapi` field. Postman creates a collection with every operation.
- Under the collection's **Authorization** tab, choose OAuth 2.0 with the authorization code grant. Use your authorizationUrl, tokenUrl, clientId, clientSecret and scopes, then log in as the test account.
- Call every operation. Check that each success response contains every field your `speakableResponse` uses, and that the error codes in `errorResponses` really occur.

7. Submit and what happens next

Upload the full file (with `private`) in the Owner Portal. Your submission moves through these states:

Status	Meaning
`submitted`	Uploaded. Automated checks run within a minute.
`checks_failed`	At least one check failed. The portal lists each problem; fix them and upload again.
`in_review`	Checks passed. An Amazon reviewer reads your tools, logs in with the test account and tries them.
`approved`	Live in the catalog. The first Alexa+ user who adds your service triggers the MCP server build, and every later user reuses it.
`rejected`	The reviewer gives a reason. Fix it and upload again.

Automated checks

- The file is valid JSON and matches the manifest schema. The `openapi` field is a valid OpenAPI 3.0/3.1 document.
- Every URL uses HTTPS and sits on `publisher.domain` or a subdomain of it.

- The hosted `/.well-known/alexa-connector.json` matches your upload, ignoring `private`.
- There are 20 operations or fewer, and each one has a unique valid operationId, a description, 2 to 5 examplePhrases, sideEffect and speakableResponse.
- Every placeholder in speakableResponse exists in the success response schema. Every payment operation has requiresConfirmation set to true.
- The token endpoint answers, and the API base URL is reachable.

8. Final checklist

☐ OAuth client created and redirect URI registered; refresh tokens enabled

☐ Test account with realistic data; its credentials are in *private.testAccount*

☐ Every operation: operationId, description, parameter descriptions, response schema, x-alexa block

☐ Payment operations confirm; write operations confirm unless the action can be undone

☐ Spoken responses read aloud and kept short

☐ Every operation tested in Postman as the test account

☐ Public copy (without *private*) live at /.well-known/alexa-connector.json

☐ Full file uploaded in the Owner Portal

Appendix: complete example (Demo Bistro)

A fictional restaurant used in our demo. This is one operation in full; the complete file (with search_menu and get_order_status) ships alongside this guide as `examples/demobistro/alexa-connector.upload.json`.


```
{
  "manifestVersion": "1.0",
  "publisher": {
    "name": "Demo Bistro",
    "domain": "demobistro.test",
    "description": "Neighborhood bistro with pickup and delivery. Search the menu, place orders
        and check on them.",
    "category": "food_and_drink",
    "logoUrl": "https://demobistro.test/assets/logo-512.png",
    "privacyPolicyUrl": "https://demobistro.test/privacy",
    "termsUrl": "https://demobistro.test/terms",
    "contactEmail": "alexa@demobistro.test"
  },
  "accountLinking": {
    "type": "oauth2_authorization_code",
    "authorizationUrl": "https://auth.demobistro.test/authorize",
    "tokenUrl": "https://auth.demobistro.test/token",
    "revocationUrl": "https://auth.demobistro.test/revoke",
    "clientId": "mcpbridge",
    "tokenAuthMethod": "client_secret_post",
    "scopes": [
      "read",
      "order"
    ],
    "pkce": true
  },
  "private": {
    "clientSecret": "REPLACE_WITH_CLIENT_SECRET",
    "testAccount": {
      "username": "reviewer@demobistro.test",
      "password": "REPLACE_WITH_TEST_PASSWORD"
    },
    "reviewNotes": "The test account has order A1042 out for delivery, so get_order_status has
        something to find."
  },
  "openapi": {
    "openapi": "3.1.0",
    "info": {
      "title": "Demo Bistro API",
      "version": "1.0.0",
      "description": "Menu search, ordering and order status for Demo Bistro."
    },
    "servers": [
      {
        "url": "https://api.demobistro.test"
      }
    ],
    "paths": {
      "/orders": {
        "post": {
          "operationId": "place_order",
          "summary": "Place an order",
          "description": "Place a delivery or pickup order, charged to the card saved on the
              user's Demo Bistro account.",
          "requestBody": {
            "required": true,
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/NewOrder"
```

```
              }
            }
          }
        },
        "responses": {
          "201": {
            "description": "Order placed",
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Order"
                }
              }
            }
          },
          "409": {
            "description": "An item is sold out"
          }
        },
        "x-alexa": {
          "examplePhrases": [
            "order a paneer wrap from Demo Bistro",
            "get me two mango lassis from Demo Bistro for pickup"
          ],
          "sideEffect": "payment",
          "requiresConfirmation": true,
          "confirmationPrompt": "Should I place this {fulfillment} order with Demo Bistro? It
              will be charged to your saved card.",
          "speakableResponse": "Done. Order {order_id} comes to {total_formatted} and should be
              with you in about {eta_minutes} minutes.",
          "errorResponses": {
            "409": "Sorry, something in that order is sold out right now."
          }
        }
      }
    },
    "components": {
      "schemas": "... NewOrder, Order, MenuItem; securitySchemes ..."
    }
  }
}
```